

Output
Best performing model : Naive Bayes
Accuracy : 1.0

P.

Expt. No.

Date.

Page No.

1a Voting Classifier - Implement a voting classifier using Decision Tree, KNN and Naive Bayes classifiers and compare the prediction results

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.ensemble import VotingClassifier
from sklearn.metrics import accuracy_score
```

```
wine = load_wine()
x = wine.data
y = wine.target
```

```
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.3, random_state=40)
```

```
dt = DecisionTreeClassifier(random_state=42)
knn = KNeighborsClassifier(n_neighbors=5)
nb = GaussianNB()
```

```
dt.fit(x_train, y_train)
knn.fit(x_train, y_train)
nb.fit(x_train, y_train)
GaussianNB()
```

```
dt_pred = dt.predict(x_test)
knn_pred = knn.predict(x_test)
nb_pred = nb.predict(x_test)
```

```
print("Decision Tree Accuracy:", accuracy_score(y_test, dt_pred))
print("KNN Accuracy:", accuracy_score(y_test, knn_pred))
print("Naive Bayes Accuracy:", accuracy_score(y_test, nb_pred))
```

```
voting_clf = VotingClassifier(
    estimators=[
        ('dt', dt)
        ('knn', knn)
        ('nb', nb)
    ]
)
```

```
voting = 'hard'
```

```
voting_clf.fit(x_train, y_train)
```

```
voting_pred = voting_clf.predict(x_test)
```

```
print("Voting Classifier Accuracy:", accuracy_score(y_test, voting_pred))
```

```
print("Decision Tree Accuracy:", accuracy_score(y_test, dt_pred))
print("KNN Accuracy:", accuracy_score(y_test, knn_pred))
print("Naive Bayes Accuracy:", accuracy_score(y_test, nb_pred))
print("Voting Classifier Accuracy:", accuracy_score(y_test, voting_pred))

print("\n Actual Labels :", y_test[:10])
print("Decision Tree :", dt_pred[:10])
print("KNN :", knn_pred[:10])
print("Naive Bayes :", nb_pred[:10])
print("Voting Classifier :", voting_pred[:10])
```

```
dt_acc = accuracy_score(y_test, dt_pred)
knn_acc = accuracy_score(y_test, knn_pred)
nb_acc = accuracy_score(y_test, nb_pred)
voting_acc = accuracy_score(y_test, voting_pred)
```

```
accuracy = {
    "Decision Tree": dt_acc,
    "KNN": knn_acc,
    "Naive Bayes": nb_acc,
    "Voting Classifier": voting_acc
}
```


Expt. No.

Date.

Page No.

```
best_model = max(accuracy, key=accuracy.get)
print("\n Best Performing Model :", best_model)
print("Accuracy:", accuracy[best_model])
```

Output

- Bagging Classifier
- estimator : Decision Tree Classifier
- Decision Tree Classifier

Bagging Classifier Accuracy : 1.0



Expt. No.

Date.

Page No.

2 Bagging Classifier - Implement Bagging using Decision Trees & evaluate the classification accuracy on the Iris dataset.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier
from sklearn.metrics import accuracy_score
```

```
iris = load_iris()
```

```
x = iris.data
```

```
y = iris.target
```

```
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3, random_state=42)
```

```
dt = DecisionTreeClassifier()
```

```
bagging = BaggingClassifier(
    estimator=dt,
```

```
    n_estimators=10,
```

```
    random_state=42
```

```
)
```

```
bagging_fit(x_train, y_train)
y_pred = bagging_predict(x_test)
accuracy = accuracy_score(y_test, y_pred)
print("Bagging Classifier Accuracy:", accuracy)
```

Q.

Output

Random forest Accuracy = 1.0

Expt. No.

Date.....

Page No.

3. Random Forest Classifier - Build a Random forest model for classification & analyze feature & importance

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

```
iris = load_iris()
```

```
x = iris.data
```

```
y = iris.target
```

```
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3, random_state=42)
```

```
rf = RandomForestClassifier(
    n_estimators=10,
    random_state=42
)
```

```
rf.fit(x_train, y_train)
```

Feature Importances

Sepal length (cm): 0.134

Sepal width (cm): 0.036

Petal length (cm): 0.42

Petal width (cm): 0.41

Expt. No.

Date

Page No.

```
y_pred = rf.predict(x_test)
```

```
accuracy = accuracy_score(y_test, y_pred)  
print("Random Forest Accuracy: ", accuracy)
```

```
print("\n Feature Importance")
```

```
for feature, importance in zip (iris.feature_names,  
rf.feature_importances_):
```

```
print(feature, ":", round(importance, 3))
```

D.

Feature importance

Sepal length (cm) : 0.134

Sepal width (cm) : 0.036

Petal length (cm) : 0.42

Petal width (cm) : 0.41



Expt No.

Date

Page No.

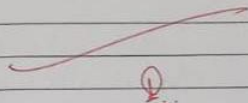
```
y_pred = rf.predict(x_test)
```

```
accuracy = accuracy_score(y_test, y_pred)  
print("Random Forest Accuracy:", accuracy)
```

```
print("\n Feature Importance")
```

```
for feature, importance in zip(iris.feature_names,  
                               rf.feature_importances_):
```

```
    print(feature, ":", round(importance, 3))
```



Output

Decision Tree accuracy : 1.0

AdaBoost Accuracy : 1.0

Expt. No.

Date

Page No.

4. AdaBoost Classifier - Implement AdaBoost on a sample dataset and compare its performance with a Decision Tree Classifier

```
from sklearn.datasets import load_iris  
from sklearn.model_selection import train_test_split
```

```
from sklearn.tree import DecisionTreeClassifier  
from sklearn.ensemble import AdaBoostClassifier  
from sklearn.metrics import accuracy_score
```

```
iris = load_iris()
```

```
x = iris.data
```

```
y = iris.target
```

```
x_train, x_test, y_train, y_test = train_test_split(  
    x, y, test_size=0.3, random_state=42)
```

```
dt = fit (x_train, y_train)
```

```
dt_pred = dt.predict(x_test)
```

```
dt_accuracy = accuracy_score(y_test, dt_pred)  
print("Decision Tree Accuracy :", dt_accuracy)
```

Expt. No.

Date

Page No.

```
ada = AdaBoostClassifier (
    n_estimators = 50
    random_state = 42
```

```
ada.fit (x_train, y_train)
```

```
ada_pred = ada.predict (x_test)
```

```
ada_accuracy = accuracy_score (y_test, ada_pred)
```

```
print ("AdaBoost Accuracy:", ada_accuracy)
```

D.

11/06/24

Expt. No.

Date

Page No.

5. Apply Gaussian Mixture model clustering on a dataset and visualize the clusters

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.mixture import GaussianMixture
```

```
iris = load_iris()
```

```
x = iris.data[:, :2]
```

```
plt.figure(figsize=(15, 4))
```

```
plt.subplot(1, 3, 1)
```

```
plt.scatter(x[:, 0], x[:, 1], color='black')
```

```
plt.title("Before GMM (Mixed Data)")
```

```
plt.xlabel("Sepal Length")
```

```
plt.ylabel("Sepal Width")
```

```
np.random.seed(42)
```

```
random_clusters = np.random.randint(0, 2, len(x))
```

```
colors = ['blue', 'red']
```

```
plt.subplot(1, 3, 2)
```

```
for i in range(len(x)):
```

```
    plt.scatter(
```

```
        x[i, 0],
```

```
        x[i, 1],
```

```
        color = colors[random_clusters[i]]
```

```
    )
```

```
plt.title("Initial Random Assignment")
```

```
plt.xlabel("Sepal Length")
```

```
plt.ylabel("Sepal Width")
```

```
gmm = GaussianMixture(
```

```
    n_components = 2,
```

```
    random_state = 42
```

```
)
```

```
gmm.fit(x)
```

```
clusters = gmm.predict(x)
```

```
plt.subplot(1, 3, 3)
```

```
for i in range(len(x))
```

```
    plt.scatter(
```

```
        x[i, 0],
```

```

x = [i, 1]
color = colors[clusters[i]]

```

```

plt.scatter(
    gmm.means[:, 0],
    gmm.means[:, 1],
    marker = 'x',
    s = 200,
    color = 'black',
    label = 'Gaussian Means'
)

```

```

plt.title("After GMM \n (Converged clusters)")
plt.xlabel("Sepal Length")
plt.ylabel("Sepal Width")
plt.legend()

```

```

plt.tight_layout()
plt.show

```

```

print("Converged :", gmm.converged_)

```

```

print("\n Gaussian Means")
print(gmm.means_)

```


Expt. No.

Date

Page No.

```
print ("\n Responsibilities (First 10 points)")
print (game.predict_proba(x) [:10])
```

Expt. No.

Date.....

Page No.